Application Development - Object Relational Mappers - 2

Table of Contents

Introduction of an ORM

Object Relational Mappers are not unique to C# and Java but exist framework for most development environments. They facilitate a way to interact with a database through code itself and with types represented within the type systems of a programming language.

Object-Relational Mappers - Internals

ORMs come in many forms and over their development have become a lot more sophisticated but at the root do require some primitives.

- Types that can be mapped to Databases
- Database Communication Protocol
- SQL Query generation mechanism
- Entity State Tracking - Similar to cache semantics

Object-Relational Mappers - Types and Tables

Tables within a database are typically represented as a Type within the source code. Fields map to the Columns. This results in either generating a types from the tables or vice-versa.

However, additional information that requires annotations to indicate special properties such as primary keys, check constraints and unique indexes.

Weak-References Translated to Strong-References

When discussing about weak-references, we need to understand that the references themselves in a database are notionally weak-reference however the enforcement of referential integrity provides a guarantee that resolution of foreign-keys to primary-keys is maintained.

With this property, we are able to map objects (which represent tuples) to other objects of the same database.

Weak References

Generally, weak references are normally a way to refer to another object or bit of data via something other than a memory address or the type itself.

Examples of weak references:

- Index to an array
- Key that maps to data structure/dictionary
- A type that represents a mechanism to retrieve the data

Strong Reference

Strong references are usually a link to directly access or retrieve the data. The risk with strong references is that they may not be loaded or not accessible (null references).

Example:

```
public class Person {
  // rest
  public DriverLicense License { get; set; }
}
```

`DriverLicense`, may or may not be loaded if have simply loaded only a `Person` object.

Object-Relational Mappers - Communication Protocol

Most ORMs will be built off a database communication layer. This allows for the application to interact with a database system. With C#, we have interacted with a database using ADO.NET, this can utilise the ODBC protocol.

Object-Relational Mappers - Communication Protocol/Query Translation

When types and tables are mapped, capabilities to generate queries when attempts to retrieve elements from a database or update, insert or remove elements from the same table.

Consider - If `public class Person` maps to table `Persons`, when retrieving entries from the `Persons` table, if we were implementing this ourselves we would map things with the following. `SELECT firstname, lastname, age FROM Persons`. We can translate that this to constructing an object.

Object-Relational Mappers - Entity State Tracking

State tracking resembles cache semantics, when data is retrieved from the database and modified by the application, the entity itself will be desynchronised from the sourced.

Entities will then be able to be updated/synchronised with the original source of information.

Object-Relational Mappers - Entity State Tracking

Entities within an ORM are tracked similarly to caches as outlined prior. They have a marker to identify if it has changed since it was retrieved. They can have the following states.

- Added - A new entity has been created and is tracked by our application but not syncrhonised.
- Unchanged - Retrieved from the database and is tracked but no values have changed
- Modified - Retrieved from the database, fields have been changed and it is now considered different from the source and may need to be synchronised.
- Deleted - The object has been deleted locally but we have not saved changes and informed the database that the object no longer exists.

ORM - Cardinality and Mapping

Using an ORM

When using an ORM, we have a few different perspectives.

- Code-First Approach
- Database-First Approach
- Mixed/Hybrid Approach

While I have constrained it to just three, it also comes down to other factors such as legacy requirements, knowledge, and maintenance.

Using an ORM - Code-First

A code first approach is one in which the types are designed within the programming language and the database schema is then derived.

This has advantages of the schema being a derivative of the codebase and not vice-versa and allows for application logic to dictate the representation.

Where it becomes problematic is when not just this codebase is using the schema/database but other applications are and the mapping can be inconsistent, it is useful applications which are isolated or new, not for existing ones.

Using an ORM - Database-First

A database first approach is one in which the relationships between tables are formed first and in-essence the database is core to the design. This is useful for existing database schemas, reimplementation of applications/upgrades or multi-application systems.

However, rigid database systems can limit what the application can do and can lead to additional mapping/workarounds to manage implementation of new features or change of requirements or transition states.

Mixed/Hybrid Approach

As outlined with the last two cases, Code-First is seemingly very start-up/new application friendly, you can break things quickly and change but is not amenable to when databases or rules are set.

A database-first approach enables existing infrastructure and data to be used however, this could be quite rigid and difficult to work with in an agile-setting.

A mixed/hybrid approach is one in which teams can consider how to best utilise what approach or what issues they may need to overcome. Prototyping could be code-first but it is acknowledged that database-first approach is needed and we can make such migrations to adhere to that.

Using an ORM - Model and Design Patterns

A common pattern which we will further explore within .NET is one called `MVC` which stands for **Model**, **View**, **Controller**. This particular pattern is different from those you have encountered prior and will be observing more formally as it is an architectural pattern.

- Model - Types which represent the data that will be used here.
- Controller - Application logic that will be used to operate on the data or query.
- View - Presentation layer, used to construct what the data should look like

Usage of Entity Framework

For this demonstration we will be focused on a code-first approach. This will be modelling the types that we want to persist in the database itself.

Usage of Entity Framework

First of all, we will be using `EntityFrameworkCore`. This allows us to have a cross-platform entity framework.

To install the package, use `dotnet add package Microsoft.EntityFrameworkCore`

Why EFCore and not regular EntityFramework

We are using EFCore because it provides the capabilities to distribute on non-Windows operating systems and makes it cheaper to deploy and maintain a server with this application.

Usage of Entity Framework

We can start by modelling classes within our code. This approach is to build types and if necessary, annotate them to know what fields we need to use. These are usually modelled as properties within a class.

```
public class UserPost
{
  public int PostID { get; set; }
  public string PostContent { get; set; }
  public int UserID { get; set; }
}
```

Usage of Entity Framework - Relationships

When modelling relationship between types, we would simply specify the types and how they are associated.

In the previous slide we had `UserPost`, this could have a `User` type that knows all posts it has made.

```
public class User
{
  public int UserId { get; set; }
  public string UserName { get; set; }
  public ICollection<UserPost> Posts { get; set;}
}
```

Usage of Entity Framework - Relationships

```
public class User
{
  public int UserId { get; set; }
  public string UserName { get; set; }
  public ICollection<UserPost> Posts { get; set;}
}
```

We use `ICollection` to indicate the collection of the data here, this forms a 1-to-Many relationship between `User` and `UserPosts`. This system here will at least either provide an empty or a collection that contains data.

Usage of Entity Framework - Relationships

However, **One-to-One** or relationships where **One-to-Many** may contain a property where `UserPost` refers to `User`.

```
public class UserPost
{
  public int PostID { get; set; }
  public string PostContent { get; set; }
  public User User { get; set; }
}
```

With these constructions, we will need to focus on how we can build these model objects.

Usage of Entity Framework

`PostID` would likely be mapped as the primary key. However, it is best to ensure it is clear what we are referring to by providing `[Key]` annotation or better yet, specifying the column. `[Column(Order=1)]` so it is clear that it is the first column.

Usage of Entity Framework

After modelling the data within our system, we will utilise `DbContext`, this will allow us to create a connection and interact with the database system.

However, inheriting from this type is also valuable as it allows us to have access to the specifics of the `DbContext` and use it as an `Information Expert` where it can hold data/cache data that could be used between different contexts.

Usage of Entity Framework

A common name is to call it `ApplicationDbContext` and specify the `connectionString` as part of the constructor.

```
public class ApplicationDbContext : DbContext
{
    private readonly string _connectionString;

    public ApplicationDbContext(string connectionString)
    {
        _connectionString = connectionString;
    }
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(_connectionString);
    }
}
```

Usage of Entity Framework

Another important method to override is `OnModelCreating(ModelBuilder modelBuilder)`. This method is to outline how we can construct our entities and their relationship.s

```
modelBuilder.Entity<T1>
  .HasOne<T2>(t2 => t2.Field)
  .WithMany(t1 => t1.CollectionField)
  .HasForiegnKey(t2 => t2.MappingKeyField);
```

Usage of Entity Framework

This translates for `UserPost` and `User` to the following.

```
modelBuilder.Entity<User>
  .HasOne<UserPost>(p => p.User)
  .WithMany(u => u.Posts)
  .HasForiegnKey(p => p.UserID);
```

This outlines how they are able to perform the mapping and load the object.

Usage of Entity Framework

To interact with the database via a context, we will need to construct a session with `using` and either retrieve or insert data into it.

```
using (var context = new ApplicationDbContext())
{

  var posts = context.UserPost
    .Where(p => p.UserId == 56)
    .ToList();
  // rest of the work
}
```

After wards, the data is composed into a list and the `UserPost` type is used as suggested by `context`.

Usage of Entity Framework

Usage of a DbContext us typically through the `using` keyword so we can immediately dispose of the connection/context after usage.

However! If you are modify data after the context has been disposed, we have **Disconnected** entities. When it is in this state, we will need to construct a new context and update.

Usage of Entity Framework

Assuming we have a `UserPost` that we have modified after the context has been disposed, we will need to attach it to a new context,

```
using (var context = new ApplicationDbContext()) {
  context.Update(modifiedUserPost);
  context.SaveChanges();
}
```

This provides the context with an idea that the state of the object was issued from a previous context and we are intending to update it when `SaveChanges()` is called.

Usage of Entity Framework - Creating an Object

We have observed briefly the usage of `DbContext` with `Update`, however lets create some data first.

```
using (var context = new ApplicationDbContext()) {
  var newUser = new User()
  {
    UserId = 20,
    UserName = "JeffRulez"
  };

  context.User.Add(newUser);
  context.SaveChanges();
}
```

The above outlines how we are able to construct a new object and save it to the database. This generates the sql needed for this.

Fitting into SDLC

Within the software development lifecycle, we can see how when given a client-brief/requirements, we would.

- Analysis - Processing and analysing the documentation and derive the requirements
- Design - How we can model it and the logic involved, the design of the application
- Implementation - After a solution has been designed, start implementing it
- Testing - As from the requirements, we should have benchmarks we need to meet
- Deployment - How this solution is to be integrated into the target system
- Maintenance - After deployment, what maintenance will look like (bug fixes, new features, etc).

Fitting into SDLC

Within SDLC, we can see how an ORM and the approach can provide an opinionated path for all components of SDLC.

This may not necessarily be the right match for all domains, however it is an effective and necessary one to understand.

Last Note

Not all problems are ones where we want to wedge an ORM into it, especially when the data is weakly referenced and may not afford what SQL provides.

Consider a case where you are using a non-SQL server, or implementing with a legacy system that is best left untouched or the queries have been optimised where an ORM will only bring convenience at best.